



ASSIGNMENT 7: CONSENSUS AND FORKS LAB

BLCH9X2 Blockchain

SAKHILE BUTHELEZI
PALESA LETHO
MAPHOKA MAHLOMOLA

University of Johannesburg
Master of Financial Engineering

1. Objective and design

We built a small Nakamoto-consensus simulator with three nodes, A, B and C, running in one process. Each node keeps its own copy of the chain, and those copies are allowed to drift apart. Instead of a single list, every node stores all the valid blocks it has seen as a *block tree*, and a fork-choice rule picks the tip it builds on. The tree is what makes switching possible: a node holding only a list would have nowhere to go when a longer competing branch turned up.

The main experiments use the longest-chain rule, as the brief allows. We also implemented heaviest cumulative work, but only as a short extension in Section 3.3. The code lives in `consensus.py`, and `consensus_lab.ipynb` runs the same code step by step.

Component	Responsibility
Tx, Block, mine()	Simple UTXO-style transactions, a Merkle root and a double-SHA-256 header. Proof of work is real but cheap: 12 leading zero bits, or roughly 4 096 hashes a block.
apply_block()	Checks each block against the ledger. Inputs must exist and be unspent, outputs cannot exceed inputs, and the coinbase cannot pay more than subsidy plus fees. A branch is accepted only if its whole history passes.
Node	Holds the block tree, an orphan pool for blocks that arrive before their parent, and a mempool. Fork choice is by length (default) or by work (extension), with first-seen, lowest-hash or random tie-breaking. In a reorganisation, transactions from dropped blocks go back to the mempool, or are flagged as reversed if they now conflict.
Network	A discrete-event gossip model. Each link has its own latency and, optionally, bandwidth, so big blocks travel slowly. We can also split the network and hold messages until it heals. A node passes every block it accepts on to its other peers.

Proof of work and fork choice

A header hash H is valid when it is below the target set by the difficulty b (the number of leading zero bits). The work in a block is the number of hashes you would expect to try before finding it:

$$H < 2^{256-b}, \quad w = 2^b.$$

For a leaf x of the tree, write $h(x)$ for its height and $W(x) = \sum_{i \in \text{path}(x)} 2^{b_i}$ for the total work along its path. The two rules then choose

$$\text{tip}_{\text{length}} = \operatorname{argmax}_x h(x), \quad \text{tip}_{\text{work}} = \operatorname{argmax}_x W(x),$$

Ties go to whichever tie-break policy is set. If every block has the same b , then $W(x) = (h(x) + 1) 2^b$, so both rules pick the same tip.

Running `python consensus.py all` replays every scenario, prints the event logs and saves them as JSON in `results/`. `analysis.py` draws the figures, and `pytest` runs eight tests, one for each claim we make below. Time is simulated and measured in seconds. We chose delays that keep the logs easy to follow; they are not meant to copy Bitcoin's ten-minute rhythm.

2. Creating and resolving a fork

We start with a shared chain G, A1, B2. At $t = 19$ s, A and C both mine a valid block on top of B2, so there are now two blocks at height 3. The links are deliberately uneven (A-B takes 1 s, B-C 3 s and A-C 6 s), which means B hears about A3 before C3.

With first-seen tie-breaking, a node sticks with the first block it received when an equally long rival arrives. The network ends up in two camps, A and B on A3 and C on C3, and stays that way until someone mines again. B does so at $t = 29$ s, building B4 on A3. When B4 reaches C at $t = 32$ s, the A3 branch is one block longer (height 4 against 3). C drops C3, adopts A3 and B4, and all three nodes agree again. C3 is now stale, and the coinbase C paid itself in it no longer exists on the ledger.

Stage (time)	A tip	B tip	C tip	Converged
common prefix ($t = 19$ s)	B2 (h = 2)	B2 (h = 2)	B2 (h = 2)	yes
fork created (before propagation) ($t = 20$ s)	A3 (h = 3)	B2 (h = 2)	C3 (h = 3)	no
after propagation: first-seen split ($t = 29$ s)	A3 (h = 3)	A3 (h = 3)	C3 (h = 3)	no
after B4: fork resolved ($t = 38$ s)	B4 (h = 4)	B4 (h = 4)	B4 (h = 4)	yes

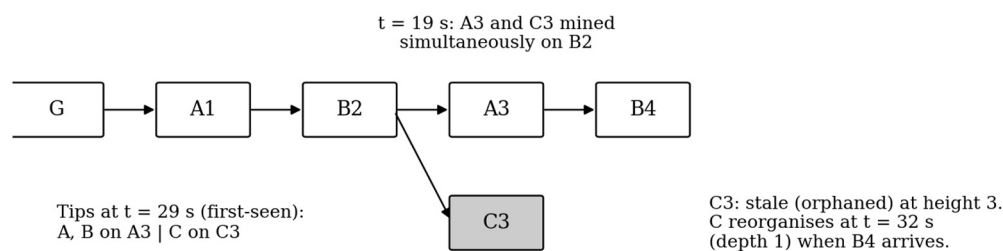


Figure 1. Block tree held by every node after resolution. The shaded block C3 is valid but off the best chain.

In the event log this appears as $t=22$ B: TIE C3 vs tip A3; keep A3 and, ten seconds later, $t=32$ C: REORG depth 1 at fork point B2: drop [C3] adopt [A3, B4]. All blocks here share one difficulty, so the heaviest-work rule would have made exactly the same choice.

3. Edge cases

3.1 Ties

Two tips of equal length make a tie. Nothing has gone wrong at that point. Both branches are valid, and the next block will decide between them. We re-ran one fork (A2 and C2, both on B1) under each tie-break policy.

Tie-break	Tips during tie (A / B / C)	After next block	Comment
first_seen	A2 / A2 / C2	all on B3	What Bitcoin Core does. Each node is stable, but the network stays divided until the next block, and one side then has a one-block reorganisation.
lowest_hash	A2 / A2 / A2	all on B3	Everyone agrees at once. The catch is that a miner who withholds blocks can publish only when its hash will win, which helps selfish mining.

Tie-break	Tips during tie (A / B / C)	After next block	Comment
random	C2 / C2 / A2	all on B3	Proposed by Eyal and Sirer (2014) to blunt selfish mining. In our run A and B switched to C2 while C switched to A2, so the split survived.

All three policies agreed after one more block. For a tie to last, the two branches would have to keep growing at exactly the same pace, and that gets less likely with every block. The flip side is that one confirmation is weak evidence: the block that confirms a payment may be one half of a tie the merchant has no way of seeing.

3.2 Delayed and out-of-order block arrival

Child before parent

Here the nodes are connected in a line (A-B-C) over 50 kB/s links. A block of size s sent over a link with latency ℓ and bandwidth β takes $\delta = \ell + s/\beta$ to arrive. A mines a 400 kB block, A2big, which needs about 8 s per hop. B then mines a small block on top of it, B3small, and the small block reaches C first, at $t = 10.1$ s. C cannot check B3small without the parent's UTXO state, so it keeps it in the orphan pool instead of throwing it away. When A2big arrives at $t = 18.0$ s, C connects the parent and then the child straight away. Decker and Wattenhofer (2013) point to exactly this effect, large blocks travelling slowly, as the main reason forks happen.

Partition and late delivery

Next we cut the network into $\{A, B\}$ and $\{C\}$. The larger side mines three blocks (A2, B3, A4) and C mines two (C2, C3), with every message across the cut held back. Once the partition heals, C sees a longer branch and reorganises two blocks deep, dropping C2 and C3. A and B receive C's blocks late, file them as a side branch and stay where they are. Late news can move a node onto a longer chain, but never back onto a shorter one.

3.3 Extension: longest chain versus heaviest work

Branch A has three easy blocks ($b = 8$, total work $3 \times 2^8 = 768$). Branch C has two hard ones ($b = 13$, total work $2 \times 2^{13} = 16384$). Under the length rule, every node settles on A4 at height 4. Under the work rule, they settle on C3 at height 3, with cumulative work of 20,481. With a fixed difficulty, as in all our other scenarios, the two rules cannot disagree. They only part ways when blocks differ in difficulty, and then counting blocks lets an attacker win with a pile of cheap ones. That is why Bitcoin compares total chainwork. In the lab, miners pick their own difficulty to make this visible; in Bitcoin, retargeting sets it.

4. Confirmation depth for a R10 000 payment

Definition

Take a payment in block x and a node whose tip is at height h_{tip} . If x is on that node's best chain, the payment has

$$z = h_{\text{tip}} - h(x) + 1$$

confirmations; otherwise it has none. The count depends on who you ask. During the fork in Section 2, a transaction that only appeared in C3 had one confirmation at C and zero at A and B.

Demonstration

C, playing the attacker, pays merchant B R10 000 from a genesis coin, and honest miner A puts the payment into block A1pay. Meanwhile C mines in private from the parent block, with a transaction that sends the same coin back to itself. The merchant ships at $z = 1$. C then releases its private branch, which is one block longer. B reorganises (depth 1), the payment is marked as reversed, and the merchant's balance falls from R10 000 to R0. We scripted C's luck here. A real attacker would have to out-mine everyone else for as long as the race lasts.

Probability

Write q for the attacker's share of hash power and $p = 1 - q$. Nakamoto (2008) approximates how far the attacker gets, while the merchant waits for z blocks, with a Poisson distribution of mean $\lambda = zq/p$:

$$P_N(q, z) = 1 - \sum_{k=0}^z \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - \left(\frac{q}{p}\right)^{z-k}\right).$$

Rosenfeld (2014) works it out exactly with a negative binomial. He also gives the attacker one block mined in advance, which is the cautious assumption:

$$P_R(q, z) = 1 - \sum_{m=0}^z \binom{m+z-1}{m} (p^z q^m - p^m q^z), \quad q < p.$$

If the attacker has no head start, it has to finish strictly ahead, and the probability becomes

$$P_S(q, z) = 1 - \sum_{m=0}^z \binom{m+z-1}{m} p^z q^m \left(1 - \left(\frac{q}{p}\right)^{z-m+1}\right).$$

We coded all three and checked them against a Monte Carlo race; Figure 2 shows how closely they match. Without the head start, the attacker's chances are roughly 3 to 6 times lower (results/depth_table.csv). For a tolerated reversal probability ε , the depth a policy needs is

$$z^*(q, \varepsilon) = \min\{z \geq 1: P_R(q, z) \leq \varepsilon\}.$$

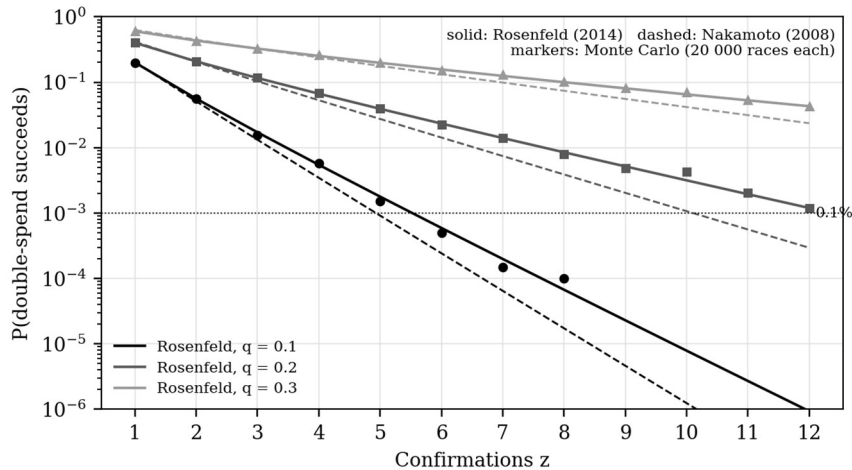


Figure 2. Probability that an attacker with hash share q eventually reverses a payment accepted after z confirmations.

Attacker share q	$z = 1$	$z = 2$	$z = 3$	$z = 6$	z^* for $\varepsilon = 0.1\%$
0.1	0.2000	0.0560	0.0171	0.0006	6
0.2	0.4000	0.2080	0.1158	0.0233	13
0.3	0.6000	0.4320	0.3262	0.1564	32

Why the answer depends on assumptions

It is tempting to compare R10 000 with the block subsidy and conclude that nobody would bother. That comparison does not settle anything. An attacker might double-spend many payments in one go, might have reasons that have nothing to do with the money, might rent hash power, or might already own mining equipment. The depth really comes down to two numbers the institution has to choose: the attacker share q it plans for and the reversal probability ε it can live with. With $\varepsilon = 0.1\%$, for example, a 10% attacker means waiting for 6 confirmations and a 20% attacker means 13.

Zero-confirmation payments face a different problem. In a *race attack*, the payer sends one transaction to the merchant and a conflicting one to the miners. No hash power is needed at all.

What depth means for R10 000

For a R10 000-equivalent payment, confirmation depth should be chosen according to the assumed attacker capability, observed network conditions, the value of the transaction and how much reversal risk the institution will accept. Accepting at zero confirmations gives the weakest protection, and each additional confirmation lowers the risk of a reorganisation further.

5. Limitations and extensions

Here are a few simplifications that should be made clear. No transactions are ever signed because that was done in the previous lecture on ECDSA. There is no retargeting of difficulty. Inventory messages, compact blocks, and peer selection have been omitted in the gossip protocol. Finally, for the double-spend problem, luck for the attacker is hardcoded rather than simulated in Monte Carlo runs. With more time we would add retargeting, headers-first sync and a selfish-mining agent. The last of these would let us measure how much each tie-break policy helps a miner who withholds blocks.

6. Reproducibility and verification

Every claim above is backed by an automated test, so a marker can re-run the whole set in about two seconds.

Test (tests/test_consensus.py)	Claim verified
test_pow_and_tamper_rejected	A block whose header was altered after mining is rejected; a valid block extends the tip.
test_fork_resolves_to_single_tip	Section 2: split after propagation, single tip after the next block, one depth-1 reorganisation.
test_first_seen_tie_keeps_split_until_next_block	Section 3.1: first-seen stays split, lowest-hash converges immediately, all policies converge after one more block.
test_orphan_pool_connects_late_parent	Section 3.2: child-before-parent is parked and later connected; partition heals to one tip.
test_heaviest_work_beats_longest	Section 3.3: the two rules select different branches.
test_double_spend_reverses_payment_at_z1	Section 4: the R10 000 payment is reversed and the merchant balance returns to zero.
test_in_block_double_spend_invalid	A block spending the same coin twice is invalid.

Test (tests/test_consensus.py)	Claim verified
test_formulas_agree_with_monte_carlo	Rosenfeld and strict formulas match simulation within 1.5 percentage points; Nakamoto matches the whitepaper table ($q = 0.1, z = 5$).

Commands: `pip install -r requirements.txt; python consensus.py all; python analysis.py; python build_docx.py; pytest -q`; or run `consensus_lab.ipynb` top to bottom.

7. Contribution statement

Member	Contribution
Sakhile	Node class, fork-choice and reorganisation logic; Sections 1 and 2; final editing.
Maphoka	Network simulator (latency, bandwidth, partitions); delayed-arrival and tie scenarios; Section 3.
Maphoka and Sakhile	Confirmation-depth analytics (Nakamoto, Rosenfeld, Monte Carlo); double-spend scenario; Section 4.
Palesa	pytest suite, figures and README; limitations and extensions.

Part B. Block relay time versus confirmation policy

In Mastering Bitcoin (Antonopoulos and Harding, 2023, Ch. 11), the blockchain is a tree of blocks linked by header hashes. Each node checks blocks for itself and follows the best branch it knows about, so a block only counts at a node once it has arrived there and passed validation. We wanted to know how the time this relay takes should affect the number of confirmations a remittance operator asks for.

The connection is forks. While a new block is still spreading, other miners are still working on the old tip, and anything they find becomes a competing block. Work spent on the branch that loses is wasted. An attacker mining in private never competes with itself, so in effect it faces a smaller honest network.

Model

Treat block arrivals as a Poisson process with mean interval T . If a block needs time d to reach the other miners, the chance that a rival appears in that window is about

$$f \approx 1 - e^{-d/T},$$

This is the simplest version of Decker and Wattenhofer's model, with one fixed delay for everyone. Our simulation has $n = 10$ equal miners, and the miner who found a block cannot compete with itself, so we compare the results with $f_n = 1 - e^{-\frac{n-1}{n}d/T}$. Forks slow honest chain growth by a factor of $(1 - f)$, which pushes the attacker's effective share up to

$$q_{\text{eff}} = \frac{q}{q + (1 - q)(1 - f)}.$$

We set the policy depth to $z^* = \min\{z \geq 1: P_R(q_{\text{eff}}, z) \leq \varepsilon\}$, with $\varepsilon = 0.1\%$ and P_R from Rosenfeld (2014). The expected wait is then

$$\mathbb{E}[\text{wait}] = \frac{z^* T}{1 - f}.$$

Once q_{eff} reaches $1/2$, no depth is enough.

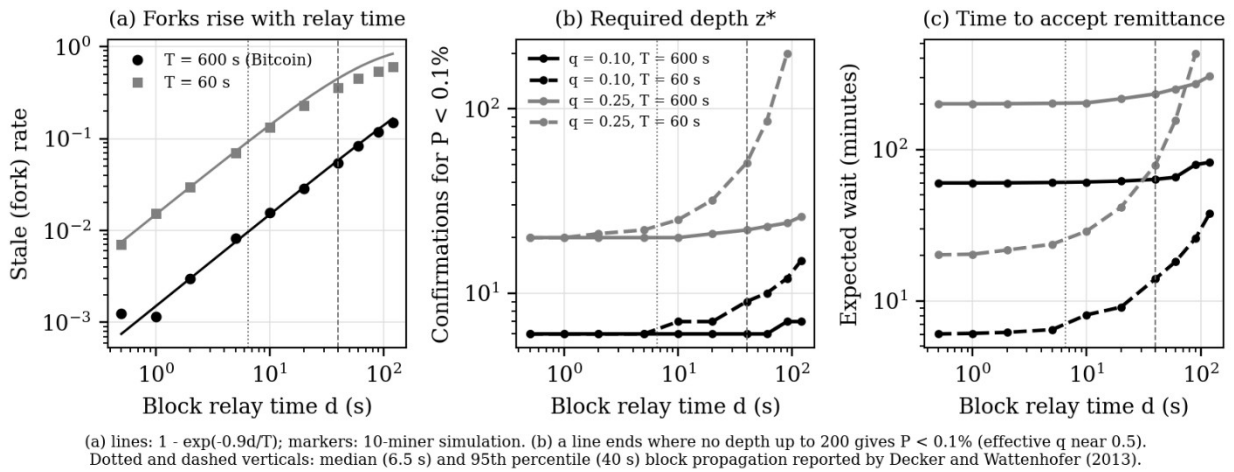


Figure 3. Relay time drives the fork rate (a), which raises the depth needed for a 0.1% reversal bound (b) and the time a remittance recipient waits (c). Solid: Bitcoin ($T = 600$ s). Dashed: a hypothetical 60-second chain.

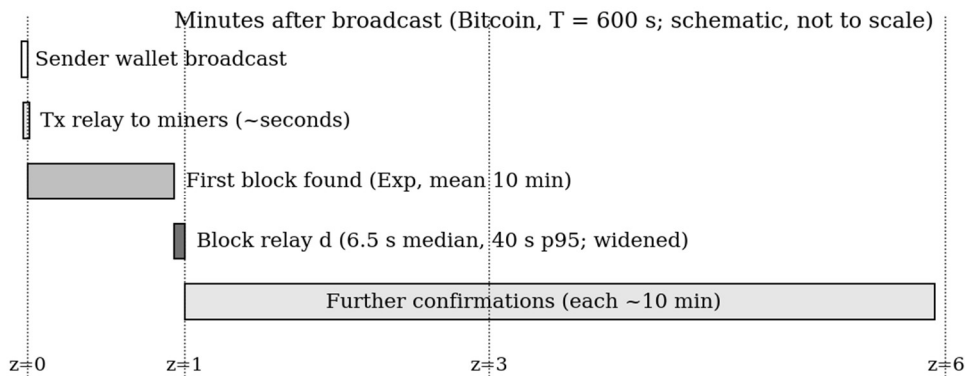


Figure 4. Where relay time sits in a Bitcoin remittance: it is a small slice of each confirmation interval, so it changes risk mainly through forks, not through waiting time.

Reading the diagram

On Bitcoin, d is tiny next to T . If relay slows from 5 to 40 seconds, the stale rate goes from 0.8% to 5.4%, yet z^* stays at 6 for $q = 0.1$ and the wait only grows from 60 to 63 minutes. A chain with 60-second blocks behaves very differently. The same slowdown lifts its stale rate from 7% to 35%, and for $q = 0.25$ it pushes z^* from 22 to 51. Slow relay down further and no depth is safe, because q_{eff} closes in on one half. Shorter block times only speed up settlement while relay stays well under T .

What this means for remittance

A policy should be stated in minutes as well as blocks, because ten-minute blocks with slow relay are safer than one-minute blocks with slow relay. Operators should watch how many stale blocks their own nodes see, since a rise is an early sign that z^* needs to go up. Faster relay, through compact block relay (BIP 152) and high-bandwidth peers, also improves security: it keeps d/T small and so keeps q_{eff} close to q . Finally, an operator should connect to plenty of well-connected peers, so that its own view of the chain is not the one lagging behind.

Part C. Memo: confirmation depth for cross-border retail payments

Recommendation

For a R10 000 equivalent transaction, the depth of confirmation should be determined based on the attacker capabilities, network environment, value of the transaction, and the degree of acceptance of reversals by the institution. Taking confirmation at zero confirmations offers the lowest level of security, and each additional confirmation reduces the possibility of reorganization. In practice, we suggest the committee (i) sets an assumed attacker share and an acceptable reversal probability for each value band, (ii) works out the Bitcoin depth from those two figures, (iii) raises the depth whenever monitoring picks up unusual fork activity, and (iv) on a Byzantine-fault-tolerant (BFT) chain, releases funds after the first committed block, as long as the validator set is known and no more than a third of validators can be faulty.

1. Why Bitcoin finality is probabilistic

According to Decker and Wattenhofer (2013), the time taken for blocks to propagate throughout the Bitcoin network is around 6.5 seconds and 12.6 seconds respectively in terms of median and mean time taken. It has a long tail since it takes 40 seconds to reach 95% of the nodes. They found out that the fork rate observed, which was 1.7%, can be explained by this latency. Because forks happen, a confirmed block can still be replaced, so every depth z leaves some reversal probability $P(q, z) > 0$. Our Part A simulation shows the same thing on a small scale. A slow block led to an orphan and an out-of-order reorganisation, and a lucky private chain reversed a R10 000 payment that had one confirmation. The authors also point out that propagation delay helps double-spenders, because different parts of the network briefly see different transactions.

2. Why BFT finality is deterministic, with conditions

Castro and Liskov's (1999) Practical Byzantine Fault Tolerance (PBFT) runs on $n = 3f + 1$ known replicas. A request commits after three phases (pre-prepare, prepare and commit), and each phase needs a quorum of $2f + 1$. Every pair of quorums shares

$$2(2f + 1) - (3f + 1) = f + 1$$

replicas, which implies that at least one honest replica belongs to both. So as long as there are less than f faulty replicas, two conflicting blocks will not commit. Liveness is dependent on time, while safety is not; this implies that once a block commits, it cannot be reorganised, and only one confirmation is necessary. However, the danger is not over. The issue is with the validators themselves since more than a third of them can conspire to commit conflicting histories. The message cost of PBFT is also quadratic to the number of validators, limiting its use to permissioned networks.

Value band (ZAR)	Tolerated reversal probability (illustrative)	Bitcoin z^* if $q = 0.05$	Bitcoin z^* if $q = 0.10$	Bitcoin z^* if $q = 0.25$	BFT chain
< R1 000	1.000%	3	4	12	1 committed block
R1 000 to R10 000	0.100%	4	6	20	1 committed block
R10 000 to R100 000	0.010%	5	8	27	1 committed block
> R100 000	0.001%	7	10	35	1 committed block

Depths use Rosenfeld (2014) with $T = 10$ minutes. The tolerances are illustrative settings for risk appetite, not recommendations. The BFT column assumes at most f of the $3f + 1$ validators are faulty.

3. Choosing a depth for R10 000

The table indicates why we cannot pinpoint an objectively safe number. If the committee assumes a 0.1% chance of reversal in the case of this band, it implies 4 confirmations, which translates to 40 minutes when facing a 5% attacker; 6 confirmations, which is around an hour when facing 10%; and 20 confirmations when facing 25%. Comparing R10 000 to the block subsidy will not solve this problem since an attacker can attempt multiple double-spends at once, has non-economic motivations, or is renting hash power. Part A also showed that a single block can be one side of a tie the merchant cannot see, so one confirmation is only a small step up from none. What the committee actually has to decide is which q and which ε it is willing to defend. The depth follows from those.

4. Customer and compliance implications

People receiving money through the corridor experience finality as a wait, so the committee should publish expected release times for each band. On Bitcoin these come to roughly $z \times 10$ minutes (Part B). The provider should also track stale blocks and peer lag on its own nodes and move to a higher band automatically when conditions get worse. None of this replaces compliance. Crypto assets are financial products under FAIS, the Financial Intelligence Centre's travel-rule directive requires originator and beneficiary details for crypto transfers, and exchange-control reporting to the South African Reserve Bank still applies to the rand leg.

5. Alternatives

In case of smaller, regular transactions, the settlement time through a payment channel like Lightning can be just seconds, with the transaction being final on the channel. The other path would be through the permissioned Byzantine Fault Tolerance-based ledger, with one-block finality and accountability built into it. In any case, the issue gets transformed. It will no longer be about waiting for how many blocks, but rather whom to trust.

Github link: <https://github.com/sakhilebut5-star/Blockchain-A7/tree/main>

References

- Antonopoulos, A.M. and Harding, D.A. (2023) *Mastering Bitcoin: Programming the Open Blockchain*, 3rd edn. Sebastopol: O'Reilly, Ch. 11.
- Castro, M. and Liskov, B. (1999) 'Practical Byzantine fault tolerance', *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI '99)*, New Orleans, pp. 173-186.
- Decker, C. and Wattenhofer, R. (2013) 'Information propagation in the Bitcoin network', *13th IEEE International Conference on Peer-to-Peer Computing (P2P 2013)*, Trento.
- Eyal, I. and Sirer, E.G. (2014) 'Majority is not enough: Bitcoin mining is vulnerable', *Financial Cryptography 2014*, LNCS 8437, pp. 436-454.
- Nakamoto, S. (2008) *Bitcoin: A peer-to-peer electronic cash system*.
- Rosenfeld, M. (2014) 'Analysis of hashrate-based double spending', arXiv:1402.2009.